C++'da Tek ve İki Boyutlu Diziler

Tanımlama, Bellek Yapısı, Fonksiyon Geçirimi ve Örnekler

Gemini Notebook

Ekim 2026

Sunum İçeriği

1. Tek Boyutlu (1D) Diziler
2. İki Boyutlu (2D) Diziler
3. Dinamik Diziler ve Modern C++

Tek Boyutlu (1D) Diziler: Tanım ve Bildirim

1D Dizi Nedir?

Bellekte **ardışık (contiguous)** alan kaplayan, aynı türdeki nesnelerin sabit boyutlu bir koleksiyonudur.

Bildirim Sözdizimi:

```
veri_tipi dizi_adi[sabit_boyut];
```

- `int numbers[5];` → Yığında (stack) 5 tamsayılık yer ayırır.
- `double salaries[100];` → 100 elemanlı ondalık sayı dizisi.
- **Önemli Kural:** Statik dizilerde boyut derleme zamanında (compile-time) bilinen bir sabit olmalıdır.

1D Dizileri Başlatma ve Elemanlara Erişim

Başlatma Biçimleri (Initialization):

```
int numbers[5] = {10, 20, 30, 40, 50}; // Tam baslatma
int data[] = {1, 2, 3};                // Boyut otomatik 3 olur
int zeros[5] = {0};                    // Tum elemanlar 0 yapilir

```

Indisleme ve Sınırlar (Indexing & Bounds):

- İndisleme **0**'dan başlar. N elemanlı bir dizide son eleman $N-1$ indisindedir.
- `numbers[0]` $\rightarrow 10$, `numbers[4]` $\rightarrow 50$
- **Sınır Kontrolü:** C++ klasik dizilerde otomatik sınır kontrolü yapmaz. `numbers[5]` erişimi tanımsız davranışa (undefined behavior) yol açar.

1D Dizilerde Bellek Yapısı ve Adres Hesabı

Doğrusal Bellek Düzeni

Dizi elemanları RAM üzerinde ardışık bellek adreslerinde tutulur.

Adres Hesaplama Formülü:

$$\text{Adres}(A[i]) = \text{Taban Adres} + i \times \text{sizeof(veri_tipi)}$$

Örnek: `int x[5]` dizisinin taban adresi 100 ve `sizeof(int)` $= 4$ bayt ise:

- `x[0]` adresi: $100 + 0 \times 4 = 100$
- `x[1]` adresi: $100 + 1 \times 4 = 104$
- `x[2]` adresi: $100 + 2 \times 4 = 108$

Dizi adı (`x`), alt indis kullanılmadığında ilk elemanın adresine (`&x[0]`) dönüşür (**pointer decay**).

1D Dizi Örneği: Fonksiyona Geçirme

```cpp
#include <iostream>
using namespace std;

void printArray(const int arr[], int size) {
    for (int i = 0; i < size; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int scores[5] = {85, 90, 78, 92, 88};
    // Diziler fonksiyona adresiyle (reference/pointer) iletilir
    printArray(scores, 5);
    return 0;
}
```

İki Boyutlu (2D) Diziler: Tanım ve Bildirim

2D Dizi (Matris) Nedir?

Satır ve sütunlardan oluşan tablo yapısıdır. Esasen bir **"diziler dizisi"** (array of arrays) yapısıdır.

Bildirim Sözdizimi:

```
veri_tipi dizi_adi[satir_sayisi][sutun_sayisi];
```

- `int matrix[3][4];` → 3 satır, 4 sütundan oluşan tamsayı matrisi (toplam 12 eleman).
- **Kullanım Alanları:** Görsel işleme (piksel matrisleri), oyun tahtaları (satranç, tictactoe), grafik komşuluk matrisleri, lineer cebir hesaplamaları.

2D Dizileri Başlatma Biçimleri

1. İç İçe Süslü Parantez (Önerilen & Okunaklı):

```
int arr[2][3] = {
    {21, 34, 42},  // Satir 0
    {22, 35, 47}   // Satir 1
};

```

2. Düz Doldurma (Tek Boyutlu Liste):

```
int arr[2][3] = {21, 34, 42, 22, 35, 47}; // Satir satir doldurur

```

3. Eksik Başlatma (Partial Initialization):

```
int arr[4][2] = {{10, 11}}; // Kalan tum elemanlar otomatik 0 olur

```

2D Dizilerde Bellek Yapısı: Row-Major Order

Satır Öncelikli Sıralama (Row-Major Ordering)

C/C++ dilinde 2D diziler bellekte **satır satır** ardışık olarak depolanır. Önce 0. satırın tüm elemanları, ardından 1. satırın tüm elemanları yerleştirilir.

2D Adres Hesaplama Formülü:

$$\text{Adres}(\text{arr}[i][j]) = \text{Taban Adres} + (i \times N_{\text{sutun}} + j) \times \text{sizeof}(\text{Tip})$$

Önbellek (CPU Cache) Performans İpucu:

- Elemanlara satır bazlı erişmek (i dışta, j içte) **cache-friendly**'dir ve çok daha hızlı çalışır.
- Sütun bazlı atlayarak erişim cache miss oranını artırır.

2D Dizi Yazdırma ve İç İçe Döngüler

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[2][3] = {
        {21, 34, 42},
        {22, 35, 47}
    };

    // Dis dongu satirlari (i), ic dongu sutunlari (j) gezer
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 3; ++j) {
            cout << arr[i][j] << "\t";
        }
        cout << endl; // Her satirdan sonra alt satira gec
    }
    return 0;
}
```

2D Dizileri Fonksiyonlara Geçirme

Önemli Kural

2D diziyi fonksiyona geçirirken **sütun boyutu** mutlaka parametrede belirtilmelidir! Derleyici bellek offset hesabı için sütun sayısına ihtiyaç duyar.

```
// Yontem 1: Standart dizi sozdizimi (Sutun boyutu 3 sart)
void printMatrix(int q[][3], int rows, int cols) {
    for (int i = 0; i < rows; ++i)
        for (int j = 0; j < cols; ++j)
            cout << q[i][j] << " ";
}

// Yontem 2: Satir isaretcisi sozdizimi
void showMatrix(int (*q)[3], int rows, int cols) {
    // *(*(q + i) + j) ile ayni adrese erisilir
}

```

Dinamik 2D Diziler (Heap Allocation)

Çalışma zamanında (runtime) boyutu belirlenen matrisler için heap bellegi kullanılır:

```
int rows = 3, cols = 4;
// 1. Isaretci dizisi olusturma
int** matrix = new int*[rows];

// 2. Her satir icin sutun dizisi ayirma
for (int i = 0; i < rows; ++i) {
    matrix[i] = new int[cols];
}

// Bellek Temizleme (Ters Sira ile!)
for (int i = 0; i < rows; ++i) delete[] matrix[i];
delete[] matrix;

```

Modern C++ Alternatifleri: `std::vector` & `std::mdspan`

1. `std::vector<std::vector<int>>` **(Modern Varsayılan):**
   - Otomatik bellek yönetimi (RAII), bellek sızıntısı riski yoktur.
   - `.at(i).at(j)` ile güvenli sınır kontrolü sunar.

```cpp
#include <vector>
// 3x4 boyutlarinda 0 ile ilklendirilmis matris
std::vector<std::vector<int>> matrix(3, std::vector<int>(4, 0));

```

2. `std::mdspan` **(C++23):**
   - Sahiplik almayan (non-owning) çok boyutlu görünüm (view). Düz 1D buffer üzerinde $1D \rightarrow 2D$ indeksleme sağlar.

Yaklaşımların Karşılaştırılması

Yaklaşım	Bellek	Dinamik Boyut	Sınır Kontrolü	Kullanım
Statik `arr[R][C]`	Stack	Hayır	Hayır	Küçük, sabit matrisler
Pointer `int**`	Heap	Evet	Hayır	STL olmadığında
`std::vector`	Heap	Evet	Evet (`.at()`)	**Modern C++ Standardı**
`std::mdspan`	View	Non-owning	Hayır	C++23 Yüksek başarım

Sık Yapılan Hatalar ve Özet

Sık Yapılan Hatalar

- **Out-of-Bounds Erişim:** Dizi sınırlarının dışına çıkma ($0 \leq i < N$).
- **Eksik Sütun Boyutu:** Fonksiyon parametresinde sütun boyutunu yazmamak.
- **Bellek Sızıntısı(Memory Leak):** Dinamik dizilerde `delete[]` çağırmayı unutmak.
- **Yanlış Döngü Sırası:** Önbellek dostu olmayan sütun-satır sırasıyla gezmek.

Teşekkürler! Soru & Cevap